

DOCUMENTO DE ARQUITECTURA

Sistema de Gestión de Pedidos — Restaurante "SaborExpress"

Fase de Ejecución — Metodología Ágil

Elaborado por: **David Santiago Fonseca Gómez**

Versión de documento: 1.0 | Fecha: 3 de octubre de 2026

Repositorio del proyecto: <https://github.com/Santi5253/Sabor-Express.git>

TABLA DE CONTENIDO

- 01 Introducción (IA)
- 02 Versión del documento
- 03 Diagrama de arquitectura
- 04 Diagrama de despliegue (MVC)
- 05 Diagrama de componentes
- 06 Buenas prácticas de desarrollo
- 07 Modelo de entidad – relación (MER)
- 08 Matriz de verificación (smoke test)

1. Introducción (IA)

NovaTech Soluciones IT S.A.S. desarrolla el **Sistema de Gestión de Pedidos "SaborExpress"** para el restaurante homónimo, en su **fase de Ejecución**. El presente documento de arquitectura describe la solución tecnológica de forma estándar: capas, despliegue bajo el patrón **Modelo–Vista–Controlador (MVC)**, componentes internos, buenas prácticas de desarrollo y el **modelo de entidad – relación (MER)**. Es la guía de construcción del sistema y la base para ejecutar el **smoke test** que verifica la coherencia de reglas, números, caracteres y requerimientos funcionales.

Objetivo

Establecer los lineamientos de arquitectura y tecnología del sistema, coherentes con la propuesta comercial, las 30 historias de usuario y el backlog por sprint, de modo que el código, la base de datos y la interfaz sigan un único estándar verificable.

Alcance del sistema

- Perfiles:** Mesero (toma de pedido), Cajero (cierre y pago) y Administrador (parametrización y reportes).
- Módulos:** autenticación y sesión, toma de pedido por mesa, cierre de pedido y pago (efectivo/débito/crédito), factura en pantalla, características esenciales: gestión de usuarios, sedes, productos, proveedores y tipos de producto, y reportes operativos exportables a Excel.
- Reglas críticas:** interfaz en inglés, transacciones ≤ 2 s, sesión por inactividad 3 min, bloqueo 5 intentos/15 min, 20 usuarios concurrentes por sede, pago único (sin combinado), sin eliminación de registros (solo edición), factura no electrónica.

Stack tecnológico **APROBADO EN PROPUESTA**

Capa	Tecnología	Versión	Rol en el patrón MVC
Front-end	HTML5 · CSS3 · JavaScript	Vanilla JS (ES2022)	Vista
Back-end	Python + FastAPI (Uvicorn)	Python 3.11+ / FastAPI 0.110+	Controlador (Routers REST)
Lógica	Services (capas de negocio)	—	Controlador / Servicios
Acceso a datos	SQLAlchemy 2.0 + Alembic	2.0+	Modelo
Base de datos	PostgreSQL + psycopg2	PostgreSQL 16	Modelo (persistencia)
Despliegue	Docker / docker-compose	Compose v2	—
Seguridad	JWT (HS256) + bcrypt + OWASP	PyJWT / passlib	Transversal
Exportación	OpenPyXL	3.1+	Transversal (reportes .xlsx)

Repositorio oficial: <https://github.com/Santi5253/Sabor-Express.git>

2. Versión del documento

Historial de versiones

Versión	Fecha	Autor	Descripción de cambios	Estado
1.0	3 de octubre de 2026	David Santiago Fonseca Gómez	Versión inicial: arquitectura, despliegue MVC, componentes, buenas prácticas, MER y smoke test.	Emitido para revisión

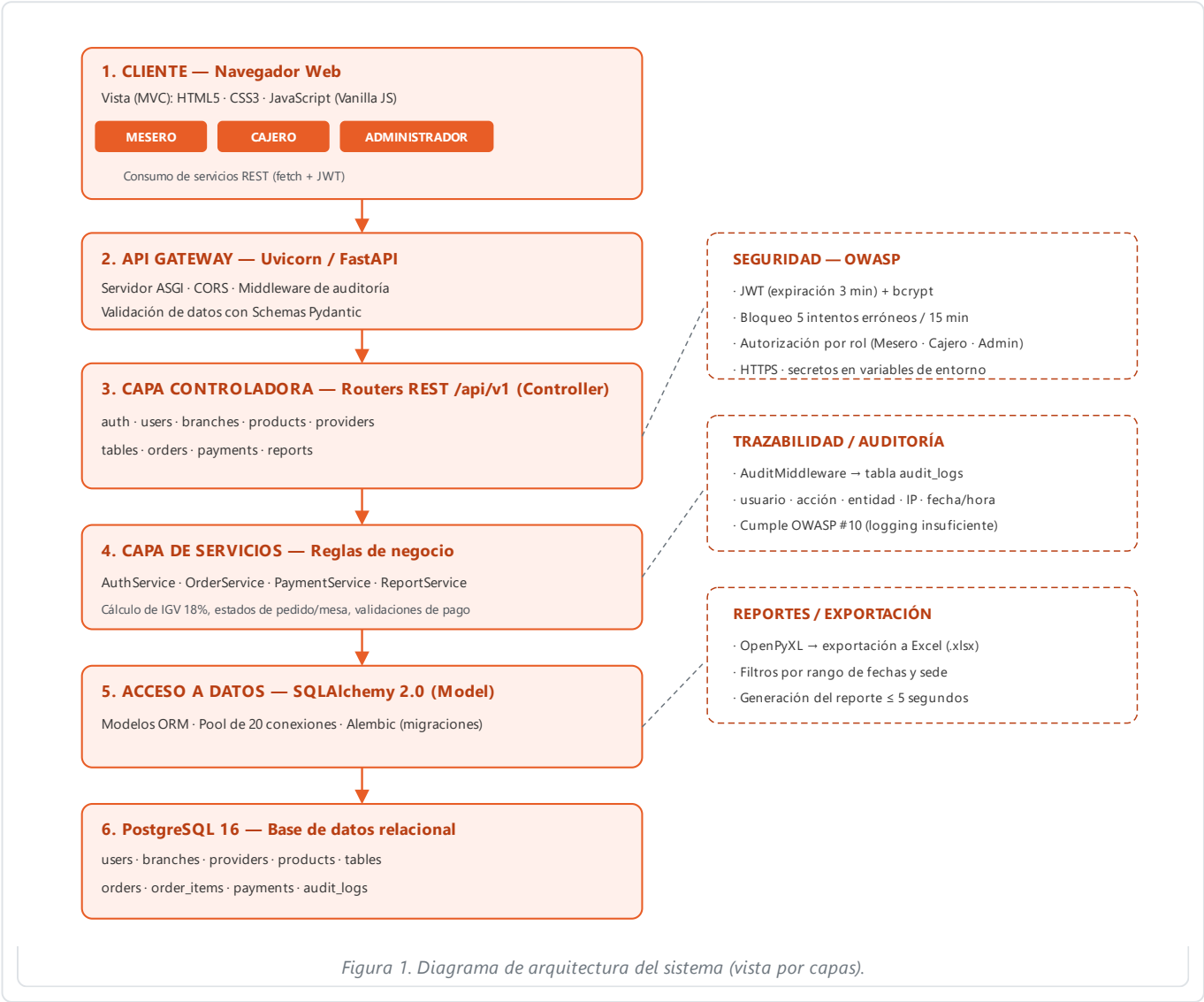
Control de cambios

Todo cambio en este documento se registrará con versión incremental, fecha, autor y detalle. La versión se alinea con los documentos del proyecto (Propuesta 1.2, Historias 1.1, BackLog 1.1).

Campo	Valor
Proyecto	Sistema de Gestión de Pedidos — Restaurante "SaborExpress"
Empresa	NovaTech Soluciones IT S.A.S
Fase	Ejecución — Metodología Ágil
Repositorio	<code>https://github.com/Santi5253/Sabor-Express.git</code>
Frecuencia de revisión	Por sprint (5 sprints), antes del cierre de cada entregable.

3. Diagrama de arquitectura

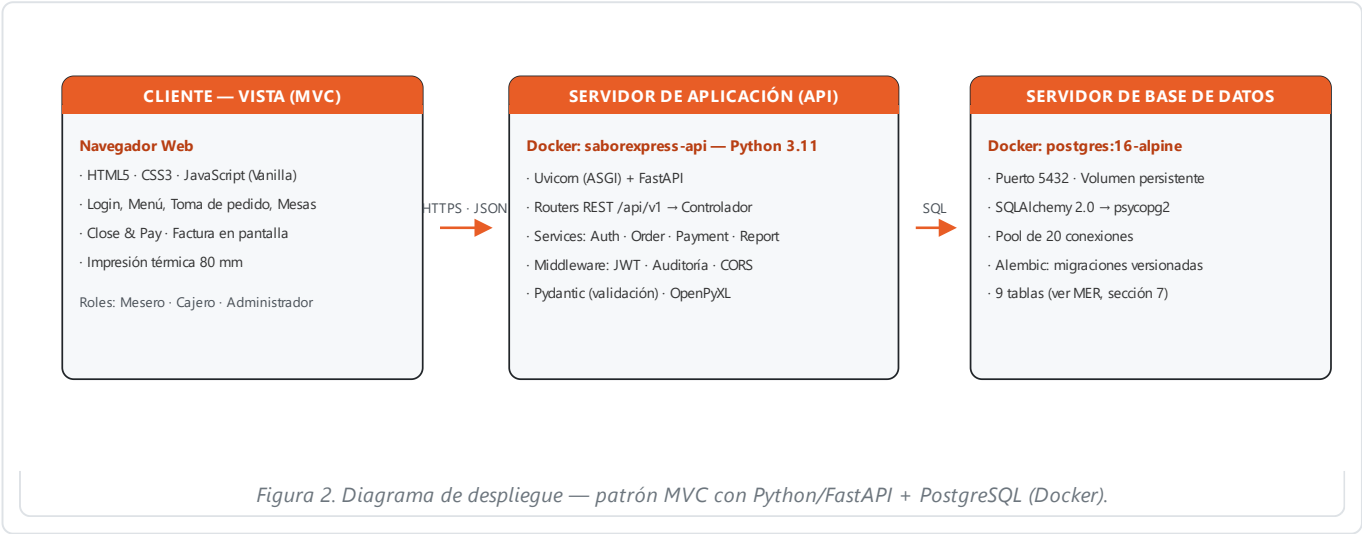
Arquitectura **multicapa (estándar)** con front-end web, API REST, capa de servicios, acceso a datos y base de datos PostgreSQL. Todas las capas se comunican mediante JSON; la seguridad (JWT), la auditoría y la exportación son componentes **transversales**.



Capa	Tecnología	Responsabilidad	Componentes
1. Cliente	HTML5/CSS3/JS	Interfaz por rol; consume REST	Login, Menú, Toma de pedido, Mesas, Close & Pay, Reportes
2. API Gateway	Uvicorn/FastAPI	Entrada REST, CORS, validación	main.py, config.py, Pydantic schemas
3. Controlador	FastAPI routers	Routing y contratos /api/v1	auth, users, branches, products, providers, tables, orders, payments, reports
4. Servicios	Python (services)	Reglas de negocio y cálculos	AuthService, OrderService, PaymentService, ReportService
5. Acceso a datos	SQLAlchemy 2.0	Modelos ORM, pool, migraciones	models/, database.py, Alembic
6. Base de datos	PostgreSQL 16	Persistencia relacional	9 tablas (MER sección 7)
Transversales	Seguridad / Auditoría / OpenPyXL	JWT+RBC, trazas, exportación	middleware, audit_logs, excel_export

4. Diagrama de despliegue (MVC)

Despliegue en **tres nodos** coherente con el patrón **Modelo–Vista–Controlador**: el navegador actúa como **Vista**, el servidor de aplicación (FastAPI) como **Controlador + Servicios** y PostgreSQL + SQLAlchemy como **Modelo**.



Nodo	Rol MVC	Artefactos	Protocolo
Navegador Web	Vista	HTML5 · CSS3 · vanilla JS, assets estáticos	HTTPS / JSON REST
Servidor de aplicación	Controlador + Servicios	saborexpress-api (Python 3.11, Uvicorn, FastAPI, Docker)	REST /api/v1 (+JWT)
Servidor de base de datos	Modelo	postgres:16-alpine, SQLAlchemy 2.0, psycopg2, Alembic	SQL (pool 20)

Mapeo MVC en el código

Capa	Ubicación en el proyecto	Ejemplos
Modelo	app/models/ + PostgreSQL	user.py, order.py, payment.py, database.py
Vista	assets/ (HTML/CSS/JS)	login.html, take-order.html, close-and-pay.html, reports.html
Controlador	app/routers/ + app/services/	orders.py, payments.py, reports.py, order_service.py

5. Diagrama de componentes

El sistema se compone de cuatro elementos principales. El diagrama interna de la base de datos varía según el motor elegido; para este proyecto se utiliza **PostgreSQL 16**, por su soporte de constraints, transacciones y concurrencia (pool de conexiones).

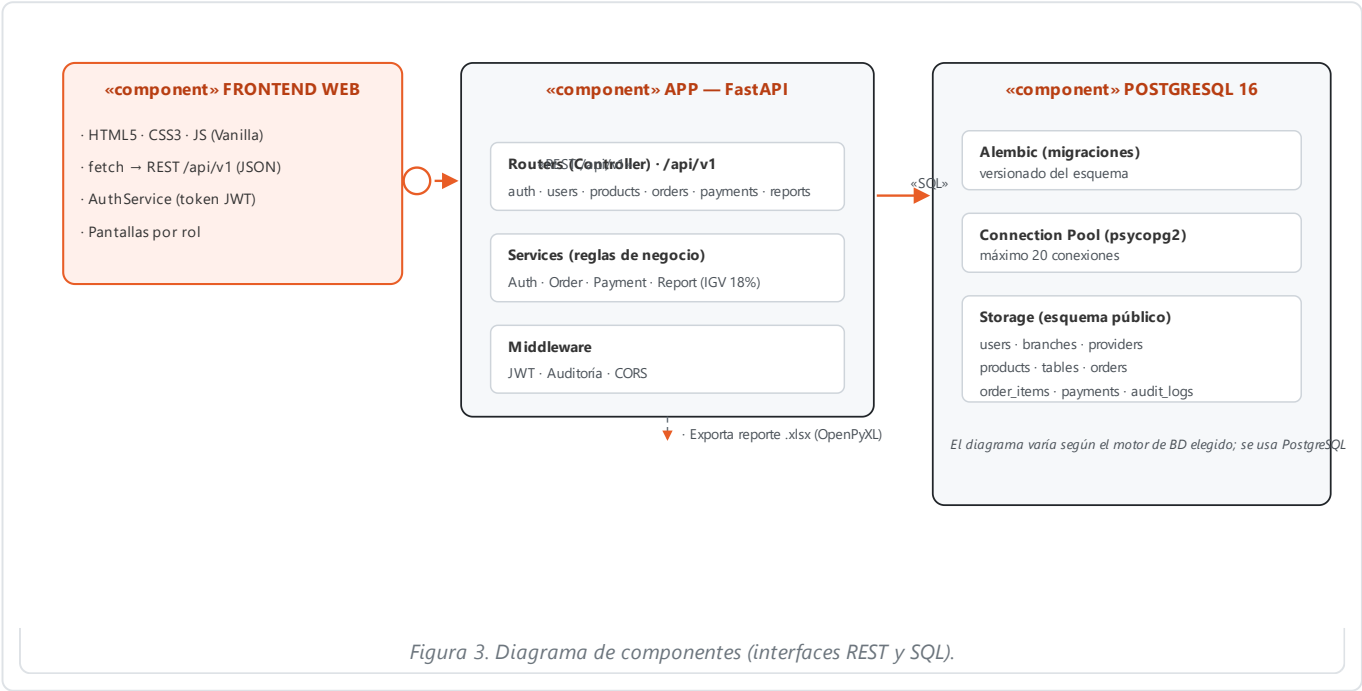


Figura 3. Diagrama de componentes (interfaces REST y SQL).

Componente	Tipo	Interfaces / Dependencias	Detalle
Frontend Web	Componente (client)	provee: consumo REST · requiere: /api/v1	HTML/CSS/JS + AuthService (JWT en localStorage)
App FastAPI	Componente (server)	provee: REST · requiere: SQLAlchemy	Routers (Controller), Services, Middleware (JWT, Auditoría, CORS), Pydantic, OpenPyXL
SQLAlchemy + Alembic	Componente (data)	provee: ORM · requiere: psycopg2	Modelos, pool (20), migraciones versionadas
PostgreSQL 16	Componente (db)	provee: SQL · requiere: conexiones	Storage, constraints PK/FK/UNIQUE, índices, 9 tablas

6. Buenas prácticas de desarrollo

Las siguientes prácticas serán implementadas y verificadas durante toda la fase de Ejecución:

Gestión de código y versionado (Git)

- Repositorio único oficial: `https://github.com/Santi5253/Sabor-Express.git` ; ramas `main` / `develop` / `feature/*`.
- Commits con mensajes descriptivos (Conventional Commits) y `.gitignore` para secretos.
- Pull Requests con revisión antes de merge y etiquetas por sprint.

Calidad y estructura del código

- PEP 8, type hints y linters/formatters (ruff, black).
- Organización por capas (routers, services, models, schemas) y docstrings en cada módulo.
- Reutilización de validaciones con Pydantic en todos los schemas.

Pruebas

- pytest: pruebas unitarias por servicio y de integración con base de datos de prueba.
- Casos de aceptación basados en las 30 HU y sus criterios (C.A.).
- **Smoke test**: al desplegar cada sprint se verifican reglas, números, caracteres y requerimientos (sección 8).

Seguridad (OWASP)

#	Práctica OWASP	Implementación
1	Inyección SQL	SQLAlchemy ORM (queries parametrizadas)
2	Autenticación rota	JWT con expiración (3 min), bcrypt
3	Exposición de datos sensibles	HTTPS, secrets en variables de entorno
4	XXE	No se usa XML en la aplicación
5	Acceso descontrolado	Middleware de autorización por rol (RBAC)
6	Configuración de seguridad	.env, CORS restrictivo, health check
7	XSS	Sanitización de inputs, content-types
8	Deserialización insegura	Validación Pydantic en cada schema
9	Componentes vulnerables	pip-audit y dependencias actualizadas
10	Logging insuficiente	AuditMiddleware → audit_logs

Base de datos

- Migraciones versionadas con Alembic (nunca alter directa).
- Constraints de unicidad (email, sede, código, NIT) e índices en FKs y búsquedas.
- Transacciones atómicas (crear pedido = pedido + estado de mesa) y pool acotado (20).
- Sin operaciones DELETE a nivel de aplicación (solo edición).

Front-end

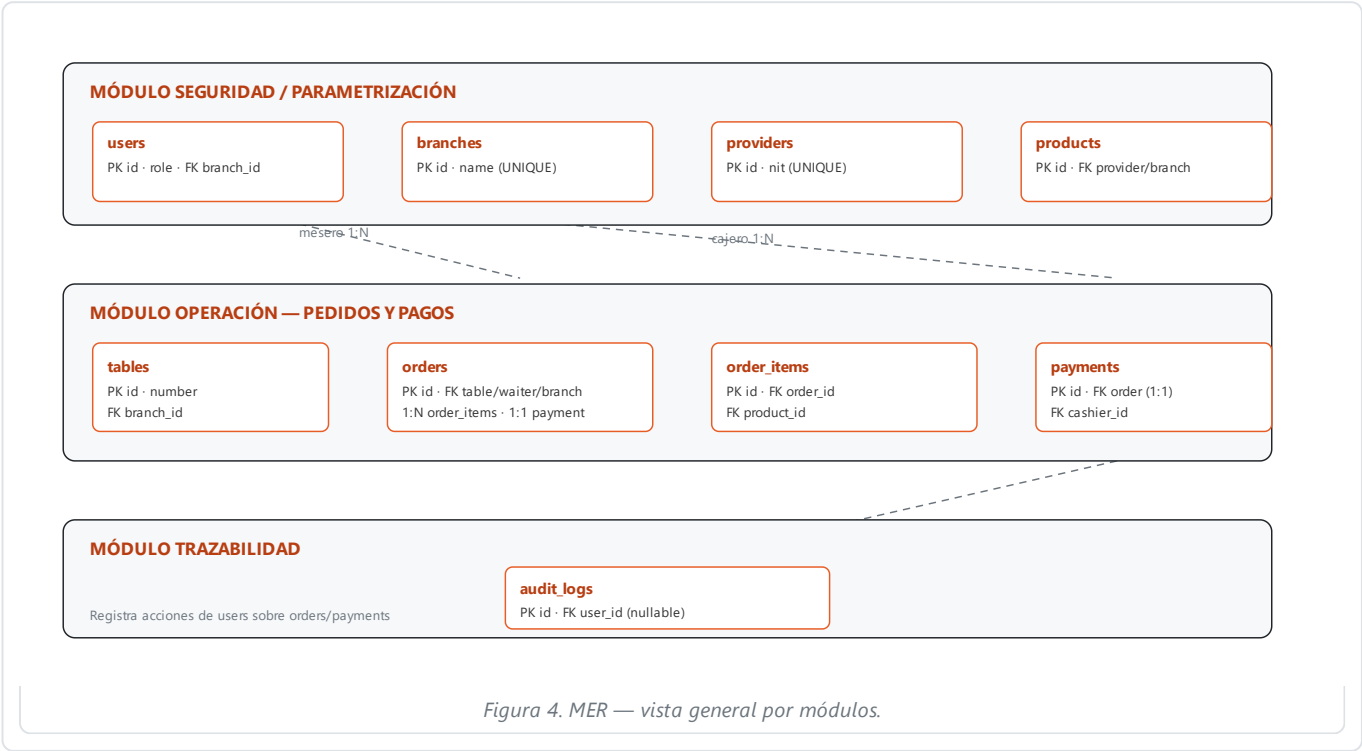
- HTML semántico, accesibilidad y diseño responsivo (PC y tablet).
- JavaScript modular (servicios con fetch), manejo de errores con toasts.
- Cierre de sesión por inactividad (3 min) gestionado en el cliente.

Despliegue y operación

- Contenedores Docker (Dockerfile + docker-compose), variables de entorno y volumen persistente.
- Health check (`/health`) e infraestructura reproducible.
- Logs estructurados y reportes auditables para trazabilidad.

7. Modelo de entidad – relación (MER)

El modelo de entidad – relación está compuesto por **9 tablas** organizadas en tres módulos. Se incluyen atributos, tipos, restricciones y relaciones. (Convención: PK = clave primaria, FK = clave foránea, 1:N / 1:1 = cardinalidad.)



Cuadro de relaciones

Entidad origen	Relación	Entidad destino	Cardinalidad
branches	posee	users / products / tables / orders	1:N
providers	abastece	products	1:N
users (Mesero)	toma	orders	1:N
tables	recibe	orders	1:N
orders	contiene	order_items	1:N
orders	genera	payments	1:1 (único método)
users (Cajero)	registra	payments	1:N
users	genera	audit_logs	1:N (FK nullable)

Detalle de las tablas (atributos y restricciones)

Tabla: users — Usuarios del sistema (Mesero, Cajero o Administrador).			
Atributo	Tipo	Restricciones	Descripción
id	SERIAL / INT	PK · AUTOINCREMENT	Identificador único del usuario.
full_name	VARCHAR(100)	NOT NULL	Nombre completo del usuario.
email	VARCHAR(150)	UNIQUE · NOT NULL · INDEX	Correo de acceso; formato válido.
password_hash	VARCHAR(255)	NOT NULL	Hash de contraseña (bcrypt).
role	ENUM	NOT NULL · Mesero/ Cajero/ Administrador	Rol de acceso y permisos (RBAC).
phone	VARCHAR(20)	NULL	Teléfono de contacto.
is_active	BOOLEAN	DEFAULT true	Estado activo/inactivo.

Atributo	Tipo	Restricciones	Descripción
branch_id	INT	FK → branches.id · NULL	Sede a la que pertenece.
login_attempts	INT	DEFAULT 0	Intentos fallidos (máx. 5).
locked_until	TIMESTAMP	NULL	Fin del bloqueo por 15 min.
last_login	TIMESTAMP	NULL	Último inicio de sesión.
created_at / updated_at	TIMESTAMP	DEFAULT now()	Fechas de auditoría de registro.

Tabla: branches — Sedes del restaurante.

Atributo	Tipo	Restricciones	Descripción
id	SERIAL / INT	PK	Identificador único de la sede.
name	VARCHAR(100)	UNIQUE · NOT NULL	Nombre de la sede (único).
address	VARCHAR(200)	NOT NULL	Dirección de la sede.
phone	VARCHAR(20)	NULL	Teléfono (opcional).
is_active	BOOLEAN	DEFAULT true	Sede activa/inactiva.
created_at / updated_at	TIMESTAMP	DEFAULT now()	Fechas de auditoría.

Tabla: providers — Proveedores asociados a productos.

Atributo	Tipo	Restricciones	Descripción
id	SERIAL / INT	PK	Identificador único del proveedor.
name	VARCHAR(100)	NOT NULL	Razón social o nombre.
nit	VARCHAR(50)	UNIQUE · NOT NULL	NIT/identificación (único).
phone	VARCHAR(20)	NULL	Teléfono de contacto.
email	VARCHAR(150)	NULL	Correo válido.
address	VARCHAR(200)	NULL	Dirección.
is_active	BOOLEAN	DEFAULT true	Activo/inactivo.
created_at / updated_at	TIMESTAMP	DEFAULT now()	Fechas de auditoría.

Tabla: products — Productos del menú (catálogo).

Atributo	Tipo	Restricciones	Descripción
id	SERIAL / INT	PK	Identificador único del producto.
code	VARCHAR(50)	UNIQUE · NOT NULL	Código del producto (único).
name	VARCHAR(100)	NOT NULL	Nombre del producto.
product_type	VARCHAR(50)	NOT NULL	Tipo de producto (select).
provider_id	INT	FK → providers.id · NOT NULL	Proveedor asociado.
price	NUMERIC(10,2)	NOT NULL · CHECK > 0	Precio (máx. 2 decimales).
is_available	BOOLEAN	DEFAULT true	Disponibilidad (Disponible/Agotado).
branch_id	INT	FK → branches.id · NOT NULL	Sede del catálogo.
created_at / updated_at	TIMESTAMP	DEFAULT now()	Fechas de auditoría.

Tabla: tables — Mesas del restaurante.

Atributo	Tipo	Restricciones	Descripción
id	SERIAL / INT	PK	Identificador único de la mesa.

Atributo	Tipo	Restricciones	Descripción
number	INT	UNIQUE · NOT NULL	Número de mesa (único).
capacity	INT	NOT NULL	Capacidad (comensales).
status	ENUM (Free/Occupied)	DEFAULT Free	Estado Libre/Ocupada.
branch_id	INT	FK → branches.id · NOT NULL	Sede de la mesa.
created_at / updated_at	TIMESTAMP	DEFAULT now()	Fechas de auditoría.

Tabla: orders — Pedidos (Abierto/Cerrado).

Atributo	Tipo	Restricciones	Descripción
id	SERIAL / INT	PK	Identificador único del pedido.
order_number	VARCHAR(20)	UNIQUE · NOT NULL	Número correlativo (PED-00042).
status	ENUM (Open/Closed)	DEFAULT Open	Estado del pedido.
table_id	INT	FK → tables.id · NOT NULL	Mesa asociada.
waiter_id	INT	FK → users.id · NOT NULL	Mesero responsable.
branch_id	INT	FK → branches.id · NOT NULL	Sede del pedido.
total	NUMERIC(10,2)	DEFAULT 0.00	Total calculado en tiempo real.
igv	NUMERIC(10,2)	DEFAULT 0.00 · 18%	IGV (18%) sobre el total.
guests	INT	DEFAULT 1	Número de comensales.
opened_at / closed_at	TIMESTAMP	now() / NULL	Apertura y cierre.
created_at / updated_at	TIMESTAMP	DEFAULT now()	Fechas de auditoría.

Tabla: order_items — Ítems de cada pedido (líneas).

Atributo	Tipo	Restricciones	Descripción
id	SERIAL / INT	PK	Identificador único del ítem.
order_id	INT	FK → orders.id · NOT NULL	Pedido al que pertenece.
product_id	INT	FK → products.id · NOT NULL	Producto agregado.
quantity	INT	NOT NULL · 1 – 99	Cantidad (mín. 1, máx. 99).
unit_price	NUMERIC(10,2)	NOT NULL	Precio unitario.
subtotal	NUMERIC(10,2)	NOT NULL	quantity × unit_price.
notes	VARCHAR(200)	NULL	Observaciones (máx. 200).
created_at	TIMESTAMP	DEFAULT now()	Fecha de registro.

Tabla: payments — Pago de un pedido (1:1).

Atributo	Tipo	Restricciones	Descripción
id	SERIAL / INT	PK	Identificador único del pago.
order_id	INT	FK → orders.id · UNIQUE · NOT NULL	Pedido (un único pago).
method	ENUM (Cash/Debit/Credit)	NOT NULL	Método de pago único.
total_amount	NUMERIC(10,2)	NOT NULL	Monto total a pagar (fijo).
amount_given	NUMERIC(10,2)	NULL (efectivo)	Monto entregado (≥ total).
change	NUMERIC(10,2)	NULL (efectivo)	Cambio calculado.
last_four_digits	CHAR(4)	NULL (tarjeta) · 4 dígitos	Últimos 4 dígitos.

Atributo	Tipo	Restricciones	Descripción
authorization_number	VARCHAR(20)	NULL (tarjeta)	Nº de autorización (máx. 20).
cashier_id	INT	FK → users.id · NOT NULL	Cajero que registró.
created_at	TIMESTAMP	DEFAULT now()	Fecha del pago.

Tabla: audit_logs — Trazabilidad de las operaciones.			
Atributo	Tipo	Restricciones	Descripción
id	SERIAL / INT	PK	Identificador único del registro.
user_id	INT	FK → users.id · NULL	Usuario que ejecutó la acción.
action	VARCHAR(50)	NOT NULL	Acción (CREATE/UPDATE/...).
entity	VARCHAR(50)	NOT NULL	Entidad afectada.
entity_id	INT	NULL	Identificador de la entidad.
details	VARCHAR(500)	NULL	Detalle de la operación.
ip_address	VARCHAR(45)	NULL	IP origen (trazabilidad).
created_at	TIMESTAMP	DEFAULT now()	Fecha/hora del registro.

8. Matriz de verificación (smoke test)

Verificación de que **todas las reglas, números, caracteres y requerimientos son lógicos y funcionales**, coherentes entre la propuesta, las historias de usuario, el backlog y el código. El smoke test se ejecuta con cada despliegue de sprint.

Reglas de validación (ejecutadas en smoke test)

Regla / requerimiento	Valor esperado	Verificación funcional	Coherencia técnica
IGV 18%	$\text{total} \times 1.18 / \text{igv} = \text{total} \times 0.18$	Pedido muestra subtotal, IGV y total	Coherente con orders.igv y HU017
Transacción ≤ 2 s	máx. 2 segundos por operación	Menú, guardar pedido, cuenta, pago	Config MAX_TRANSACTION_SECONDS = 2
Cierre por inactividad	3 min (aviso a los 2 min)	Sesión expira automáticamente	JWT ACCESS_TOKEN_EXPIRE_MINUTES = 3
Recordar sesión	extiende token a 7 días	Checkbox "Remember session"	Config REFRESH 7 días
Bloqueo de cuenta	5 intentos / 15 min bloqueo	Mensaje de cuenta bloqueada	login_attempts y locked_until
Concurrencia	20 usuarios simultáneos / sede	Horas pico estables	Pool de 20 conexiones (max_overflow=0)
Cantidad por ítem	mín. 1 · máx. 99	Modal de cantidad + observaciones	VALIDACIÓN order_items.quantity
Observaciones	máx. 200 caracteres	Campo notes por ítem	order_items.notes VARCHAR(200)
Precios	2 decimales · positivos	Validación numérica	price NUMERIC(10,2) CHECK > 0
Tarjeta	últimos 4 dígitos obligatorios	Campo maxlength=4	payments.last_four_digits CHAR(4)
Autorización tarjeta	obligatorio · máx. 20	Campo maxlength=20	payments.authorization_number VARCHAR(20)
Contraseña	≥ 8 · mayúscula · número · especial	Validación al registrar/ingresar	password_hash (bcrypt)
Pago único por pedido	sin pago combinado	Solo un método de pago	payments.order_id UNIQUE (1:1)
Efectivo	monto entregado $\geq$ total · cambio	Aviso si el monto es menor	amount_given / change en payments
Estados de pedido	Open $\rightarrow$ Closed	Cerrado no se modifica	orders.status ENUM
Estados de mesa	Free $\leftrightarrow$ Occupied	Cambio automático al guardar/cerrar	tables.status ENUM
Sin eliminación de registros	solo editar (UPDATE)	Sin DELETE en el sistema	Endpoints PUT; sin endpoint DELETE
Unicidad	email · sede · código · NIT	mensajes de duplicado	UNIQUE constraints en BD
Reporte	Generate Report ≤ 5 s	filtros fecha y sede obligatorios	ReportService + índices
Exportación	Descarga .xlsx ≤ 3 s	Nombre SaborExpress_Report_...	OpenPyXL (Openpyxl 3.1+)
Factura	en pantalla (no electrónica)	Impresión térmica 80 mm	Factura vista en pantalla
Idioma de interfaz	inglés	Front-end en inglés	HTML/CSS/JS (Vanilla)
Autenticación por rol	Mesero/Cajero/Admin	Redirección y permisos	Middleware por rol (RBAC)
Trazabilidad	cada operación registrada	audit_logs con usuario/IP/fecha	AuditMiddleware

Conclusión

✓ Todos los requerimientos resultan **lógicos y funcionales**: los números (IGV 18%, 2 s, 3 min, 5/15, 20 usuarios, 99 ítems, 4 dígitos, 20 caracteres, 2 decimales), las reglas de caracteres (contraseña y correo), los estados de negocio (pedido Abierto/Cerrado, mesa Libre/Ocupada) y las reglas funcionales (pago único, sin eliminación, factura en pantalla, reportes exportables) son coherentes con las 30 historias de usuario, el backlog por sprint y el modelo de datos descrito en la sección 7.